AI Agents for Recovering and Modernizing Legacy Software

Project Proposal • Sebastian Rodriguez • October 29, 2026

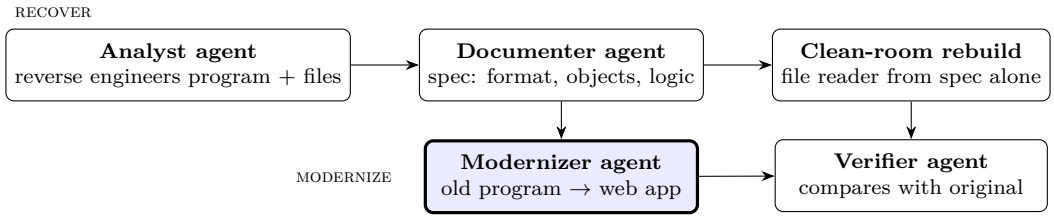
Motivation

Software disappears all the time. Programs stop running when their platform dies, their company stops selling them or their source code is lost, and the work people built with them goes too. Emulators keep some of it alive as museum pieces, but nobody can extend or maintain software that only runs inside an emulator. This project builds a general system of AI agents that takes an abandoned program, recovers how it works, and carries it forward into modern, maintainable software. The first case study is **HyperCard** (Apple, 1987), a tool that let non-programmers build interactive "stacks" of cards scripted in HyperTalk; Apple stopped selling it in 2004, and its stacks now run only in emulators of old Macs.

Goal and approach

The goal is a reusable multi-agent pipeline: the input is a legacy program plus files made with it; the output is a specification of how it works and a modern version a developer can maintain. The steps are the same for every target; only the tools change with the kind of software.

Reverse engineering is the agents' main tool. Abandoned software rarely comes with source code or documentation, so the agents work from the shipped program itself. Through an MCP server they disassemble and decompile binaries, inspect file formats byte by byte, and observe the program running in an emulator. A core question is how well an agent can use these tools on its own, a task that today needs a human specialist.



1. **Recover.** The analyst agent studies the program and its data files; the documenter turns the findings into a spec (file formats, object model, script behavior). A separate agent builds a file reader from the spec alone to test that it is accurate (clean-room approach).
2. **Modernize.** The modernizer maps the program onto a modern architecture and translates its logic into readable code a person can edit. For HyperCard: cards become pages, fields become state, HyperTalk becomes JavaScript.
3. **Verify.** The verifier runs the original in an emulator next to the modern version, repeats the same actions in both, and reports differences back to the modernizer.

Evaluation

Tested on HyperCard stacks of growing difficulty, then on a second program of a different kind: **spec accuracy** (test files the clean-room reader opens correctly), **visual fidelity** (screenshot comparison per screen), **behavioral fidelity** (scripted actions with the same result in both), **maintainability** (can a person add a feature to the output), and **generality** (how much of the pipeline carries over unchanged to the second program).

Scope and risks

The deliverable is the reusable pipeline, proven on a few representative stacks (not every feature). HyperTalk has many edge cases, so translation may stay partial; the fallback is reporting which features convert reliably. Targets are limited to abandoned software or software whose license allows analysis, and no original code is copied into the output.

Tools and prompt history

Claude Code connected to a reverse-engineering MCP server (REA: decompilation, file-format analysis, GDB/Frida instrumentation), plus emulators. All work runs from one project folder, so every session transcript is saved under `~/.claude/projects/`; a `PROMPTS.md` log keeps every prompt in English and marks the turns that changed the project's direction, for the presentation.